

Trabalho Prático 1: polinômios esparsos

Grupos de **3 alunos**.

Entrega até **15 de setembro de 2026**.

1. Motivação

Um polinômio de grau um milhão pode ter três termos. Guardar um coeficiente para cada grau possível gastaria 8 MB para representar algo que cabe em 100 bytes. Neste trabalho, o grau pode chegar a 10^{18} , e aí a mesma conta dá 8 milhões de terabytes. A escolha de uma estrutura de dados adequada pode ser a diferença entre o programa existir e não existir.

O mesmo problema aparece longe da álgebra. Um serviço de streaming precisa saber quais músicas cada usuário ouviu. Com 10 milhões de usuários (10^7) e um milhão de músicas (10^6), a tabela completa teria 10^{13} células. Como um usuário médio ouviu apenas algumas centenas de faixas, mais de 99% dela está vazia. Ocupar tantos espaços em branco é tão ineficiente que a tabela inteira não caberia na memória de nenhuma máquina.

Uma das possíveis soluções para esse problema é utilizar uma representação esparsa, e a receita é sempre a mesma: guarde apenas o que não é zero e pague o preço na hora de percorrer e combinar. É o que sustenta bibliotecas científicas populares, como a `scipy.sparse` do Python, e é executado quando um programa de álgebra simbólica expande uma expressão. Ao fim deste trabalho vocês terão desenvolvido uma versão pequena disso.

Guardar só o que não é zero é a parte fácil. Mas, a parte que será mais avaliada neste trabalho é decidir **como** guardar da melhor maneira. Cada estrutura ganha em algumas operações e perde em outras, e não há nenhuma que ganhe em todas as operações possíveis. Por isso, este enunciado não diz qual usar: a escolha é de vocês, e defendê-la é metade do trabalho.

2. O que o programa faz

O programa lê comandos da entrada padrão e escreve na saída padrão até `FIM`. Cada polinômio tem um nome e envolve uma única variável, x .

Comando	Efeito	Saída
DEF nome k	define nome (do polinômio) a partir das k linhas seguintes, cada uma com coef grau	nenhuma
SOMA A B R	R recebe $A + B$	nenhuma
PROD A B R	R recebe $A * B$	nenhuma
ADD A c g	soma a A o termo de coeficiente c e grau g	nenhuma
ESCALA A c	multiplica todo coeficiente de A pelo inteiro c	nenhuma
COEF A g	coeficiente de grau g em A	o coeficiente
REMOVE A g	apaga de A o termo de grau g	nenhuma
REMOVEMENOR A	apaga de A o termo de menor grau	nenhuma
GRAU A	maior grau com coeficiente não nulo	o grau
IMPRIME A	escreve A em ordem canônica	o polinômio
IMPRIMEINV A	escreve A em ordem crescente de grau	o polinômio
LIBERA A	devolve a memória de A; o nome fica indefinido	nenhuma
FIM	encerra	nenhuma

A e B precisam existir. R não precisa: se ainda não existir, é criado; se já existir, seu conteúdo anterior é descartado.

Os três nomes de polinômios podem se repetir. SOMA A B A é válido e sobrescreve A.

Limites

Grandeza	Limite
grau	0 a 10^{18}
coeficiente na entrada	-10^3 a 10^3
termos em um DEF	100 000
polinômios vivos ao mesmo tempo	512
tamanho de um nome	32 caracteres
comandos em uma execução	200 000

Os dados de teste garantem que nenhum coeficiente intermediário ultrapassa 2^{62} , então `long` basta em toda parte. Repare no limite de grau antes de decidir como guardar os termos.

3. Formato da saída

A ordem canônica é de grau decrescente.

Cada termo é escrito como $c \cdot x^g$, com o sinal incluído no coeficiente. Os termos são separados por um espaço. Não há exceção: o coeficiente 1 aparece, o grau 0 aparece, o grau 1 aparece.

$3 \cdot x^2 - 5 \cdot x^1 + 7 \cdot x^0$

O polinômio nulo é representado por 0.

Um polinômio nunca contém termo de coeficiente zero. Se dois termos de mesmo grau se cancelam, o termo deixa de existir.

4. Casos que o seu programa precisa tratar

Situação	Comportamento
DEF com graus repetidos	os coeficientes se somam
coeficiente que zera	o termo some
DEF nome 0	polinômio nulo
GRAU de polinômio nulo	-1
COEF de grau ausente	0

Situação	Comportamento
REMOVE de grau ausente	nada acontece
ADD de grau que já existe	os coeficientes se somam, como em DEF
ADD com coeficiente 0	nada acontece
REMOVEMENOR de polinômio nulo	nada acontece
ESCALA A 0	A passa a ser nulo
redefinir um nome já usado	o conteúdo anterior é descartado
LIBERA seguido de DEF no mesmo nome	válido

5. Exemplos

5.1 Operações básicas

Entrada	Saída
DEF A 3	$3x^2 - 5x^1 + 7x^0$
3 2	$7x^0 - 5x^1 + 3x^2$
-5 1	2
7 0	-5
IMPRIME A	
IMPRIMEINV A	
GRAU A	
COEF A 1	
FIM	

5.2 Cancelamento e polinômio nulo

Entrada	Saída
DEF Z 2	0
5 3	-1
-5 3	0
IMPRIME Z	
GRAU Z	
COEF Z 3	
FIM	

5.3 ESCALA por zero

Entrada	Saída
DEF P 2	$3x^4 + 8x^1$
3 4	0
8 1	-1

```

IMPRIME P
ESCALA P 0
IMPRIME P
GRAU P
FIM

```

5.4 Produto e alias

Entrada	Saída
DEF B 2	$1*x^2 - 1*x^0$
1 1	$2*x^1$
1 0	
DEF C 2	
1 1	
-1 0	
PROD B C P	
IMPRIME P	
SOMA B C B	
IMPRIME B	
FIM	

5.5 ADD e REMOVEMENOR

Entrada	Saída
DEF A 2	$5*x^4 - 7*x^2 + 3*x^1$
5 4	$5*x^4 - 7*x^2$
3 1	$5*x^4$
ADD A 7 2	
IMPRIME A	
ADD A -3 1	
IMPRIME A	
REMOVEMENOR A	
IMPRIME A	
FIM	

6. A decisão que é de vocês

A escolha de qual estrutura de dados usar é do grupo e é o que o relatório defende. O que torna a escolha difícil é que **as operações puxam para lados diferentes**, e nenhuma estrutura de array/lista ganha em todas as operações. Exemplos:

- **COEF favorece o array.** Num array ordenado dá para saltar direto ao meio e achar um grau por busca binária. Numa lista só resta percorrer.
- **ADD depende da ordem em que os termos chegam.** Se chegam do menor grau para o maior, cada um entra numa ponta; se chegam do maior para o menor, entra na outra. Uma estrutura que é barata numa ponta costuma ser cara na outra, e um array paga deslocamento onde uma lista só religa ponteiros.

Cada escolha também tem um custo de memória por termo, que não é igual entre elas.

Meçam antes de concluir. A intuição da análise assintótica acerta em algumas dessas comparações e erra em outras, porque percorrer memória contígua é muito mais rápido do que seguir ponteiros espalhados, e isso não aparece na notação *big O*. O relatório precisa mostrar que vocês perceberam esses conflitos e os mediram, não apenas que escolheram algo que funciona.

7. Desempenho

Não há limite de tempo nem de memória que valha nota. Um programa correto e lento não perde ponto na correção automática. Mas a diferença entre uma solução pensada e uma improvisada é grande, e medi-la faz parte do trabalho. Para medir é preciso uma entrada grande e grande demais para digitar.

Por isso vocês entregam também um **segundo programa, separado do principal**, chamado `gerador.c`. Ele não resolve nada e não conhece nenhum polinômio: só imprime, na saída padrão, uma sequência de comandos com o perfil da tabela abaixo. São poucas dezenas de linhas de código. Para gerar uma entrada e medir o desempenho, vocês podem fazer a seguinte sequência de comandos:

```
./gerador > entrada.txt  
time ./seu_programa < entrada.txt
```

Este perfil é o mesmo para todos os grupos, de propósito, para que os números de relatórios diferentes possam ser comparados.

Repare por que `c` nunca é 0: um único `ESCALA A 0` zera o polinômio, e todos os `ESCALA` seguintes passam a não ter nada a fazer. A medição perderia o sentido.

Parâmetro do gerador	Valor
termos no <code>DEF</code> inicial	50 000
graus	sorteados sem repetição em <code>[0, 500 000]</code>
coeficientes	sorteados em <code>[-1000, 1000]</code>
<code>ESCALA</code> em sequência	20 000, com <code>c</code> sorteado entre <code>-1</code> e <code>1</code> , nunca 0
depois	um <code>IMPRIME</code> , um <code>GRAU</code> e um <code>SOMA</code> com um segundo polinômio gerado do mesmo jeito

O `gerador.c` faz parte da entrega.

8. O que entregar

Num arquivo `.zip`:

Item	O que é	Observação
o programa principal, em <code>.c</code> e <code>.h</code>	interpreta os 11 comandos da seção 2. É o trabalho	compila sem avisos com <code>gcc -Wall -Wextra -O2</code>
a implementação da estrutura de dados escolhida, em <code>.c</code> e <code>.h</code>	implementa a estrutura de dados escolhida (TAD)	
<code>gerador.c</code>	programa à parte, que só imprime a entrada de teste da seção 7	poucas dezenas de linhas
<code>Makefile</code>	compila os dois	
<code>relatorio.pdf</code>	4 a 8 páginas	

O programa principal lê da entrada padrão e escreve na saída padrão. Nada de menu, nada de mensagem de boas-vindas, nada de arquivo temporário.

O `gerador.c` não conhece polinômio algum e não é avaliado quanto à corretude. Ele existe só para vocês conseguirem medir a própria solução.

Como entregar

- No e-Disciplinas, na atividade **TRABALHO 1**.
- **Só um integrante submete**, de preferência, o primeiro nome do relatório.
- O `.zip` se chama `T1_<numeroUSP>.zip`, com o número de quem submete.
- Os arquivos vão na **raiz** do `.zip`, sem pasta em volta.

9. Perguntas Frequentes

Estas respostas valem como parte do enunciado.

1. **Qual padrão de C?** C11 ou posterior, definido com `-std=` no `Makefile`, que também precisa de `-Wall -Wextra`.
2. **Podemos usar alguma biblioteca?** Só a biblioteca padrão de C. Sem dependências externas.
3. **Podemos dividir em vários arquivos?** Sim, e é uma boa ideia.
4. **Precisamos validar a entrada?** Não. A entrada de teste é sempre bem formada e respeita os limites da seção 2.

5. **Podemos escrever na saída de erro?** Sim. A correção compara apenas a saída padrão, então as mensagens de depuração no `stderr` não atrapalham.
6. **Somos obrigados a usar uma das estruturas da seção 6?** Não. Aquelas são exemplos para mostrar o conflito, não um cardápio. Vocês podem utilizar quaisquer estruturas de **listas**, desde que a escolha seja devidamente justificada no relatório.
7. **Quais são os casos de teste?** Os da seção 5 e os da seção 4 estão garantidos. A bateria completa tem mais, mas não é divulgada.
8. **O relatório pode ser em inglês?** Pode, desde que o grupo inteiro concorde.

10. Estrutura do relatório

Cinco itens, nesta ordem.

Item	O que precisa ter
Capa	instituição, disciplina, título do trabalho e nome completo e número USP de cada integrante do grupo.
Projeto da solução	qual estrutura, e como cada operação foi implementada
Justificativa	tabela de custos das operações nas estruturas consideradas (min. 3), e a justificativa da estrutura escolhida para a implementação final
Medição	os tempos que vocês mediram com o gerador, e o que eles mostram
Ponto fraco	uma carga de trabalho em que a escolha de vocês se sai mal
Casos de teste	os casos que vocês criaram e o que cada um testa

10-1 Comparações entre listas

Para cada um dos quatro tipos de listas na tabela a seguir, qual é o tempo de execução assintótico do pior caso para cada operação em conjuntos dinâmicos apresentada na lista?

	não ordenada, simplesmente ligada	ordenada, simplesmente ligada	não ordenada, duplamente ligada	ordenada, duplamente ligada
SEARCH(L, k)				
INSERT(L, x)				
DELETE(L, x)				
SUCCESSOR(L, x)				
PREDECESSOR(L, x)				
MINIMUM(L)				
MAXIMUM(L)				

Figura 1: tabela “Comparações entre listas” do livro-texto da disciplina (Cormen et al.). A tabela de custos do relatório de vocês tem esta estrutura, porém cada coluna é uma estrutura de dados que vocês consideraram utilizar no trabalho.

A tabela de custos vale mais do que a implementação. Um grupo que escolheu a estrutura mais simples e explicou bem o que ela custa tira nota melhor do que um grupo que escolheu a mais rápida, sem saber explicar por quê.

11. Avaliação

Bloco	Peso
Corretude	3,0
Implementação e uso de memória	2,0
Relatório	5,0

Atraso desconta 1,0 ponto por dia.

Trabalhos são comparados entre si e com material público por meio de uma ferramenta automática de similaridade. A identificação de plágio resultará em nota ZERO nos trabalhos de todos os envolvidos.